LIVRABLE 0 - README - Détection Automatique de Fraude

Ce projet est une infrastructure complète de détection de fraude en temps réel, utilisant un **ensemble LightGBM + XGBoost** comme modèle de classification et **Apache Airflow** comme orchestrateur de pipeline.

Installation rapide

1. **Environnement Conda** :

```
conda create -n fraud_detection python=3.10 -y
conda activate fraud_detection
pip install -r requirements.txt
```

2. **Configuration de la base de données Neon** :
   - Créer un projet sur neon.tech
   - Copier `.env.example` vers `.env` et renseigner vos propres credentials :

```
cp .env.example .env
```
- Exécuter le schéma initial dans la console SQL de Neon :
```bash
psql $DATABASE_URL -f database/init.sql
```

3. **Lancement de l'infrastructure (Docker)** :

```
docker-compose up -d
```

```
Cela démarre :
- **airflow-db** : PostgreSQL local (métadonnées Airflow uniquement)
- **Airflow Webserver** sur le port `8081` (identifiants : admin / admin)
- **Airflow Scheduler** pour l'orchestration des DAGs
- **Dashboard Streamlit** sur le port `8501`
> Les données métier (transactions, prédictions) sont hébergées sur **Neon**
(cloud), configuré via `DATABASE_URL` dans `.env`.
```

4. **Execution du pipeline via Airflow** :
   - Accéder à l'interface Airflow : `http://localhost:8081`
   - Déclencher le DAG `ingestion_historique` pour charger les données CSV
   - Déclencher le DAG `entrainement_modele` pour entraîner le modèle ensemble
   - Activer le DAG `worker_temps_reel` pour la simulation en continu

- Le DAG `rapport_quotidien` s'exécute automatiquement chaque matin à 8h

5. **Execution manuelle (sans Airflow)** :

```
# Charger les variables d'environnement
source .env

# Charger les données historiques
python src/ingestion/load_historical.py data/fraudTest.csv

# Entraîner le modèle
python src/ml/train.py

# Lancer le worker temps réel
python src/ingestion/worker.py data/fraudTest.csv

# Lancer le tableau de bord
streamlit run src/app/main.py
```

Architecture

Le détail des choix techniques et le schéma d'infrastructure sont disponibles dans :

- `ARCHITECTURE.md` : Schéma de flux et justifications techniques.
- `PIPELINE_QUALITY.md` : Qualité des données et observabilité du pipeline.

Fonctionnalités

- **Ingestion temps réel** : Simulation d'une API de paiement avec stockage dans **Neon PostgreSQL** (cloud), orchestrée par Airflow.
- **Détection proactive** : Inférence ML immédiate sur chaque transaction entrante via l'ensemble LightGBM + XGBoost.
- **Système d'alerte** : Notification dans la console et les logs Airflow en cas de fraude.
- **Reporting quotidien** : DAG Airflow générant un rapport CSV et un résumé des fraudes chaque matin.
- **Supervision métier** : Interface Streamlit pour visualiser les KPI (taux de fraude, volume) et explorer les transactions.

Modèle

Le modèle actuel est un **ensemble LightGBM + XGBoost (soft voting)**, entraînable en local sur le dataset complet (~555k transactions) en moins d'une minute.

Pourquoi ce duo ? LightGBM et XGBoost sont tous deux du gradient boosting mais avec des stratégies de construction d'arbres opposées : LightGBM croît *leaf-wise* (plonge vers les zones les plus informatives, idéal pour détecter les fraudes atypiques) et XGBoost croît *level-wise* (arbres équilibrés, stabilise les prédictions sur la majorité des transactions normales). Leurs erreurs étant partiellement décorrélées, la moyenne de leurs probabilités (*soft voting*) donne un score plus robuste sur les cas limites — précisément ceux où la détection de fraude est la plus critique.

Features utilisées :

- Montant de la transaction (`amount`)
- Catégorie de dépense (`category`)
- Heure de la transaction (`hour`)
- Jour de la semaine (`day_of_week`)
- Statistiques comportementales par client (`client_tx_count`, `client_avg_amount`, `client_amount_ratio`)
- Volume de transactions par marchand (`merchant_tx_count`)

> **Approche initiale** : TabPFN v2 via API PriorLabs a été évaluée en premier. Abandonnée en raison de la limite de 100M cellules/jour (quota épuisé en un seul entraînement) et de la dépendance à une API externe. Code archivé dans `archive/train_tabpfn_v1.py`.

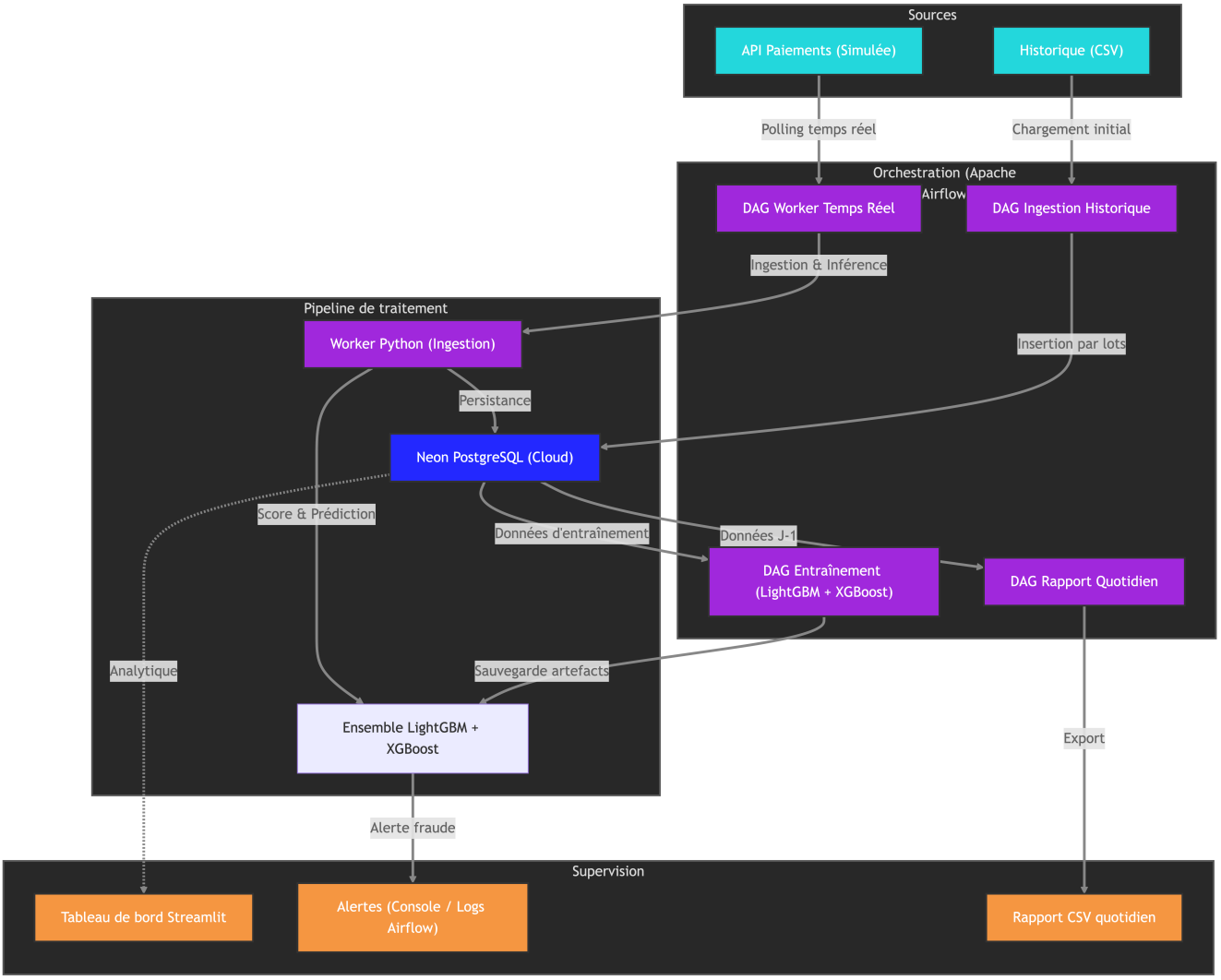
Orchestration Airflow

DAG	Planification	Description
`ingestion_historique`	Manuel	Charge le CSV historique en base
`entrainement_modele`	Hebdomadaire	Entraîne/ré-entraîne l'ensemble LightGBM + XGBoost
`worker_temps_reel`	Toutes les minutes	Ingestion + prediction en continu
`rapport_quotidien`	Chaque jour a 8h	Rapport des fraudes de la veille

LIVRABLE 1 - Architecture et structure du projet

1. Schéma d'architecture (flux de données)

Ce schéma décrit le flux de données depuis l'ingestion jusqu'à la visualisation, illustrant un pipeline de détection temps réel orchestré par Airflow.



2. Justification des choix technologiques

- **Ensemble LightGBM + XGBoost — soft voting (Modèle ML)** : Deux modèles de gradient boosting aux stratégies complémentaires. LightGBM croît *leaf-wise* (spécialiste des patterns rares, capte les fraudes atypiques) ; XGBoost croît *level-wise* (arbres équilibrés, robuste sur la majorité des transactions normales). La moyenne de leurs probabilités (*soft voting*) réduit la variance des prédictions sur les cas limites. Le déséquilibre de classes (≈ 0,38 % de fraudes) est géré nativement via `scale_pos_weight`, sans SMOTE. Entraînable en local sur le dataset complet en < 1 min. — *Approche initiale TabPFN v2 abandonnée : limite de 100M cellules/jour et dépendance API incompatibles avec l'itération rapide. Code archivé dans* `archive/train_tabpfn_v1.py`.
- **Apache Airflow (Orchestration)** : Planification, monitoring et gestion des dépendances entre les tâches du pipeline. Remplace les scripts manuels par des DAGs reproductibles et observables via l'interface web.
- **Neon PostgreSQL (Base de données)** : Base PostgreSQL serverless cloud. Mêmes propriétés ACID qu'un PostgreSQL local, avec en plus : disponibilité managée, backups automatiques,

scaling automatique et accès distant sécurisé (SSL). Sert de backend pour les données applicatives ET les métadonnées Airflow.

- **Streamlit (Visualisation)** : Mise en oeuvre rapide d'un tableau de bord interactif en Python pour le reporting métier.
- **Docker Compose (Infrastructure)** : Garantit la reproductibilité de l'ensemble de l'infrastructure (Airflow, Dashboard) quel que soit l'environnement de déploiement. La base de données étant externalisée sur Neon, Docker Compose ne gère plus le stockage.

3. Performances du modèle (dernière évaluation — 31/03/2026)

Entraînement sur **444 597 transactions** (80 % du dataset), évaluation sur **55 575 transactions** dont **214 fraudes** (taux réel : ≈ 0,38 %).

Métrique	Valeur
AUC-PR *(Area Under Precision-Recall Curve)*	**0.8827**
Recall fraude (seuil 0.5)	**0.87**
Précision fraude (seuil 0.5)	0.63
F1 fraude (seuil 0.5)	0.73
Temps d'entraînement (local)	≈ 43 s

Lecture : l'AUC-PR de 0.88 indique un fort pouvoir discriminant sur l'ensemble des seuils, malgré un déséquilibre de classes de 258:1. Le recall de 0.87 signifie que 87 % des fraudes sont détectées à seuil neutre (0.5), au-dessus de l'objectif métier fixé à 75 %.

> **Comparaison avec l'approche initiale (TabPFN v2)** : AUC-PR 0.63 → 0.88 (+39 %), recall 0.75 → 0.87, évaluation sur 4 fraudes → 214 fraudes (fiabilité statistique réelle).

LIVRABLE 2 - Qualité des données et observabilité du pipeline

Ce document détaille les procédures de contrôle qualité et de monitoring implémentées dans le pipeline de détection de fraude pour répondre aux exigences du Bloc 3 de la certification AIA.

1. Contrôles qualité automatisés (Data Quality)

Le pipeline intègre des mécanismes de validation à chaque étape pour garantir l'intégrité des prédictions :

Ingestion (Source -> Brut)

- Validation du schéma : Le script `worker.py` s'assure que les champs obligatoires (`transaction_id`, `amount`, `category`, `timestamp`) sont présents avant toute insertion.
- Conversion de types : Conversion forcée des types (ex : timestamp en `datetime`, montants en `float`) pour éviter les erreurs de calcul en aval.
- Orchestration : Airflow détecte et relance automatiquement les tâches échouées (retries configurables par DAG).

Évaluation du modèle

- Jeux de validation et test stratifiés : La séparation train/val/test (80/10/10) conserve le taux de fraude réel (≈ 0,38 %) dans chaque split. Le jeu de test final contient 214 fraudes — suffisant pour que les métriques (recall, précision, AUC-PR) soient statistiquement fiables.
- Optimisation du seuil sans leakage : Le seuil de décision est optimisé sur le jeu de validation (215 fraudes), puis évalué sur le jeu de test (214 fraudes) pour éviter tout surapprentissage du seuil.
- Guard F2 : Si le seuil optimisé dégrade le F2-score (β=2, favorise le recall) par rapport au seuil neutre 0.5, le pipeline revient automatiquement à 0.5 — garantissant qu'une optimisation mal contrainte ne peut pas régresser les performances en production.

Transformation (Ingénierie des features)

- Gestion des valeurs manquantes : Le pipeline de prédiction vérifie l'absence de valeurs nulles sur les features critiques (`amount`, `category`).
- Référentiel temporel : Synchronisation de l'heure système avec l'heure de la transaction pour garantir la cohérence des features temporelles (`hour`, `day_of_week`).
- Cohérence des transformateurs : Le scaler et l'encodeur de catégories sont sauvegardés avec le modèle pour garantir un pré-traitement identique entre entraînement et inférence.

2. Observabilité et surveillance

Le pipeline est piloté par trois outils complémentaires :

Monitoring opérationnel (Airflow)

L'interface web Airflow fournit :

- État des DAGs : Visualisation en temps réel du statut de chaque tâche (succès, échec, en cours).
- Historique des exécutions : Traçabilité complète de chaque run avec durée, logs, et résultat.

- **Alertes d'échec** : Notification automatique en cas d'échec d'une tâche (configurable par email ou Slack).
- **Dépendances** : Graphe visuel des dépendances entre les tâches du pipeline.

Monitoring métier (Streamlit)

Le tableau de bord fournit une vue consolidée de la "santé" des données :

- **Taux de fraude anomal** : Une augmentation soudaine du taux de fraude peut indiquer un problème de dérive du modèle ou une corruption des données d'entrée.
- **Volume de transactions** : Permet de vérifier que le pipeline d'ingestion ne subit pas d'interruption (baisse de cadence).

Monitoring technique (Logs et alertes)

- **Traçage des prédictions** : Chaque prédiction est enregistrée dans la table `predictions` de **Neon PostgreSQL** avec la version du modèle utilisée (`lgbm-xgb-ensemble-v1`), permettant un audit a posteriori en cas de faux positifs.
- **Logs Airflow** : Les détections de fraude sont journalisées dans les logs de chaque exécution de DAG, accessibles via l'interface web.

3. Gestion des erreurs et reprise (Error Correction)

- **Reprises automatiques (Airflow)** : Chaque DAG est configuré avec un nombre de tentatives (`retries`). En cas d'échec réseau ou base de données, la tâche est automatiquement relancée.
- **Blocs Try/Except** : La boucle du worker est protégée contre les erreurs, assurant la résilience du flux.
- **Exécution unique** : Le DAG `worker_temps_reel` est configuré avec `max_active_runs=1` pour éviter les doublons et les conditions de concurrence.
- **Logs d'erreurs** : En cas d'échec d'une insertion, l'erreur est capturée et journalisée sans arrêter le pipeline global (principe de tolérance aux pannes).

4. Reporting et traçabilité

- **Rapport quotidien automatisé** : Le DAG `rapport_quotidien` génère chaque matin un fichier CSV contenant toutes les transactions et fraudes de la veille, archivé dans `data/rapports/`.
- **Versionnement du modèle** : Chaque prédiction est associée à la version du modèle (`lgbm-xgb-ensemble-v1`), permettant de tracer l'évolution des performances.
- **Historique complet** : L'interface Airflow conserve l'historique de toutes les exécutions, offrant une vue d'ensemble de la fiabilité du pipeline.

5. Conformité à la gouvernance

Toutes les données ingérées respectent la politique de gouvernance définie (anonymisation des données sensibles comme le numéro de carte bancaire, qui est haché ou tronqué).